Consignes générales pour les TDs

FAITES BIEN LES SCHEMAS DEMANDES, **CE N'EST PAS OPTIONNEL** ET VOUS AIDERA À VISUALISER ET COMPRENDRE LES ALGORITHMES

POUR LES EXERCICES COMPORTANT DE LA REDACTION DE CODE EN C, **UTILISEZ UN EDITEUR DE TEXTE OU UN IDE**, AFIN DE POUVOIR RECUPERER VOS TRAVAUX POUR LES TPs.

Les exercices précédés d'un symbole losange ◆ sont obligatoires.

TD 4 : Arbres binaires, partie I

Compléments de cours

Exercice 1) Compter le nombre de nœuds présents dans un arbre

On souhaite écrire une fonction pour compter le nombre de nodes présents dans un arbre binaire.

En vous inspirant de l'exemple donné en cours pour le calcul de la hauteur, écrivez une fonction récursive, à qui l'on fournit un **`t_node *`** en paramètre (pour le traitement d'un node quelconque), puis une fonction qui appelle la première pour la racine d'un arbre, de type **`t_tree`**

quelle est alors la complexité de cette fonction de comptage ?

Exercice 2) comprendre les parcours et les structures d'arbre

Dessinez les schémas correspondants aux deux programmes suivants (on suppose que la fonction createNode() retourne un **`t_node *`**), et que la fonction **`createTree()`** fournit un arbre vide

Programme1 :

```
int main(void)
{
    t_tree tree;
    t_node *temp, *nouv;

    tree = createTree();

    nouv = createNode(47);
    tree.root = nouv;

    nouv = createNode(12);
    tree.root->left = nouv;

    nouv = createNode(-3);
    tree.root->right = nouv;
    temp = tree.root->right;

    nouv = createNode(11);
    temp->left = nouv;

    nouv = createNode(47);
```

```
    temp->right = nouv;

    return 0;
}
```

programme 2

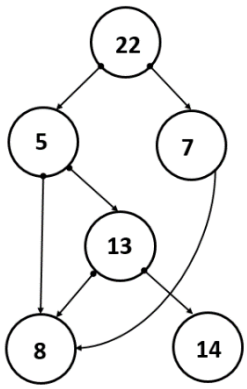
```
int main(void)
{
    t_tree tree;
    t_node *temp, *nouv;

    tree = createTree();

    nouv = createNode(47);
    tree.root = nouv;

    nouv = createNode(12);
    tree.root->left = nouv;
    tree.root->right = nouv;

    temp = tree.root->right;

    nouv = createNode(11);
    temp->left = nouv;
    temp->right = nouv;

    return 0;
}
```

Indiquez ce que donnent les parcours préfixe, infixe et postfixe, pour la structure de données créée dans le programme 2

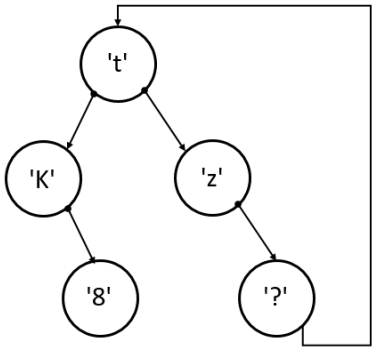
Exercice 3)

Soient les 'arbres' suivants:

A)



B)



Question 1) Pour chacun de ces ‘arbres’, donnez le résultat des parcours :

Préfixe; Infixe; Postfixe.

Question 2) La question est : l’ ”arbre” A peut-il être qualifié d’arbre ?

À partir de la représentation de l’ “arbre” A, calculez sa hauteur, comme calculée récursivement sur un arbre classique. (une feuille a une hauteur de 0).

Cette hauteur correspond-elle à la définition de la hauteur classique d’un arbre binaire ?

Question 3) Combien la fonction récursive de l’exercice 1 compte-t-elle de nœuds ?

Question 4) Combien de nœuds ont été créés en mémoire?

Question 5) Si nous insérons un nœud en tant que fils du nœud qui stocke la valeur ‘8’, combien de nœuds comptera la fonction treeNodeCount() ?

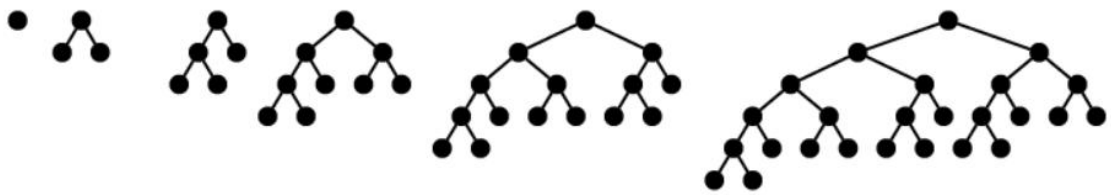
Question 6) Est-ce compatible avec la définition d’un arbre binaire?

Exercice 3) Arbres de Fibonacci

Soit la suite d’arbres FT: définie ainsi :

> FT_0 est un arbre avec un seul nœud ,de valeur 0
>
> FT_1 est un arbre dont la racine est nœud de valeur 1, et dont les fils gauche et droit sont des feuilles de valeur 0
>
> $\forall\, n \geq\ 2, FT_n$ est un arbre dont la racine stocke la valeur n, le sous arbre gauche de cette racine est FT_{n-1} , son fils droit est FT_{n-2}

<u>Illustration (INRIA)</u>



Question 1) Visualisez les arbres FT_0 à FT_3 en indiquant les valeurs stockées dans les nœuds – A quoi correspondent ces valeurs ?

Question 2)

Remplissez le tableau suivant :

n for FT_n	L'arbre est-il strict?	Est-il complet ?	Est-il parfait?
0			
1			
2			
3			
4			
5			

Exercise 4) Rechercher une valeur dans un arbre

Une fois de plus, la récursivité est de mise. On cherche à savoir si une valeur est stockée dans un arbre. Si oui, on veut récupérer **le sous arbre** dont la racine est le nœud qui stocke la valeur cherchée, en utilisant une fonction **SeekValue()**

Étape 1) Trouver un `t_node *` dans un arbre , à partir d'un pointeur **p_n**

p_n est le pointeur à partir duquel on fait la recherche**.**

1.1) Si la valeur cherchée n'est pas dans le sous arbre pointé par **p_n**, quelle valeur doit être retournée ?

1.2) Quelle sont alors les conditions de fin de récursivité ?

1.3) Quel est le cas général de récursivité ?

<u>Question 2)</u> Écrivez la fonction `seekValue()`

<u>Question 3)</u> à partir de cette fonction, écrivez la fonction `isValueInTree()`, qui fait cette même recherche, mais se contente de retourner vrai/faux (1/0)